Session 11 - Duplicate Functions, Default Params, Function Expression, this

Naveen AutomationLabs | 2026 Playwright Batch G1

1. Duplicate Functions (No Overloading)

JavaScript does NOT support function overloading. If you define multiple functions with the same name, only the last definition survives.

```
function search() {
console.log('hello search1');
}
function search(productname) {
console.log('hello search2', productname);
}
function search(productname, price) {
console.log('hello search3', productname, price);
}

search('imac', 1000, 300);
// Output: hello search3 imac 1000
// The last 'search' function overwrites the previous ones.
```

Extra arguments are simply ignored. Missing arguments become undefined.

2. Default Parameters

You can set default values for function parameters. If no argument is passed for that parameter, the default value is used.

```
function login(username, password, role = 'admin', status = 'active') {
console.log(username, password, role, status);
}

login('naveen@gmail.com', 'naveen@123');
// Output: naveen@gmail.com naveen@123 admin active

login('naveen@gmail.com', 'naveen@123', 'seller');
// Output: naveen@gmail.com naveen@123 seller active

login('kiran', 'kiran@123', null, 'inactive');
// Output: kiran kiran@123 null inactive
```

Passing null explicitly overrides the default. null is a valid value. Only when you skip the argument entirely (or pass undefined) will the default be used.

3. Function Expression

A function expression assigns a function to a variable. Unlike function declarations, they are NOT hoisted.

```
let cart = function addToCart(productName) {
console.log('add to cart', productName);
```

```
return true;
};

// cart is the variable name (function expression name)
// addToCart is the actual function name

let n1 = cart('macbook pro'); // works
console.log(n1); // true

// addToCart('macbook'); // ReferenceError - can't call by actual name

console.log(cart.name); // 'addToCart' (returns actual function name)
```

You call the function using the variable name (cart), not the actual function name (addToCart).

Hoisting difference:

```
// This will throw ReferenceError:
// initDriver('chrome'); // called before definition

let initDriver = function (browserName) {
console.log('browser: ' + browserName);
};

// Function declarations ARE hoisted, function expressions are NOT.
```

4. Objects with Functions (Methods)

Objects can have functions as properties. These functions are called methods.

```
let user = {
name: 'mukul',
age: '30',
dept: 'QA',

coding() {
console.log('user is coding....');
},
travel() {
console.log('user is travelling....');
}
};

console.log(user.name); // mukul
user.coding(); // user is coding....
user.travel(); // user is travelling....
```

5. The 'this' Keyword

Inside an object method, 'this' refers to the current object. Use it to access other properties and methods of the same object.

```
let loginPage = {
username: 'naveen@gmail.com',
password: 'naveen@123',
role: 'admin',

login() {
console.log('login to app using: ' + this.username);
```

```
this.resetPwd();
},
resetPwd() {
console.log('reset pwd');
this.logout();
},
logout() {
console.log('logout from app');
}
};

loginPage.login();
// Output:
// login to app using: naveen@gmail.com
// reset pwd
// logout from app
```

'this' is essential when one method needs to call another method in the same object, or when you need to read a property of the same object.

6. Function Chaining and Recursion

Functions can call other functions (chaining). A function calling itself is called recursion.

```
function login() {
console.log('login to app');
search();
}
function search() {
console.log('perform search');
addToCart();
}
function addToCart() {
console.log('add to cart');
}
login(); // login -> search -> addToCart
```

Be careful: if functions call each other in a circle (login -> search -> addToCart -> login), it creates infinite recursion and crashes with 'Maximum call stack size exceeded'.